Summary

This runbook provides some hints and details on how to restore a namespace from a database backup and disk snapshots.

> This is NOT the preferred method to restore a deleted project. For that, please see the Deleted Project Restoration Runbook

It is probably a good idea to read that runbook before executing anything in this one, since this runbook assumes that you've made a PITR database restore instance following the process laid out there.

Make the database instance read/write

By default the PITR database restore instance is read/only. To make it read/write:

```
gitlab-ctl stop postgresql
rm /var/opt/gitlab/postgresql/data/standby.signal
gitlab-ctl start postgresql
```

Restore the snapshots

Change the zone and type options to your desired destination

```
gcloud compute --project=gitlab-restore disks create file-09-stor-gprd-data-08-15-2020 --sou
gcloud compute --project=gitlab-restore disks create file-10-stor-gprd-data-08-15-2020 --sou
gcloud compute --project=gitlab-restore disks create file-11-stor-gprd-data-08-15-2020 --sou
gcloud compute --project=gitlab-restore disks create file-12-stor-gprd-data-08-15-2020 --sou
gcloud compute --project=gitlab-restore disks create file-13-stor-gprd-data-08-15-2020 --sou
gcloud compute --project=gitlab-restore disks create file-14-stor-gprd-data-08-15-2020 --sou
gcloud compute --project=gitlab-restore disks create file-15-stor-gprd-data-08-15-2020 --sou
gcloud compute --project=gitlab-restore disks create file-16-stor-gprd-data-08-15-2020 --sou
gcloud compute --project=gitlab-restore disks create file-17-stor-gprd-data-08-15-2020 --sou
gcloud compute --project=gitlab-restore disks create file-18-stor-gprd-data-08-15-2020 --sou
gcloud compute --project=gitlab-restore disks create file-19-stor-gprd-data-08-15-2020 --sou
gcloud compute --project=gitlab-restore disks create file-20-stor-gprd-data-08-15-2020 --sou
```

Mount all shards at once

First attach all of the shards created above in the GCP console

Add this to the end of `/etc/fstab`

```
UUID=98a22450-f02e-4ee7-ab76-97ff490bc90a   /mnt/file-09    ext4    defaults    0 0
UUID=ba8c6f55-c8d9-4e3c-a955-4c49d5e0efb6   /mnt/file-10    ext4    defaults    0 0
UUID=01b2c8c8-5d7f-48a3-b143-1178387007bc   /mnt/file-11    ext4    defaults    0 0
UUID=ea60a66c-b279-45cf-8896-02e92b9404e1   /mnt/file-12    ext4    defaults    0 0
UUID=99edc798-1ee8-43d9-abd1-1a4b849d0b0e   /mnt/file-13    ext4    defaults    0 0
```

```
UUID=3ccc3991-cda8-45b1-9dc4-b504692d196e   /mnt/file-14    ext4    defaults    0 0
UUID=377695e8-3b6b-4d07-ad05-358bb930b6f0   /mnt/file-15    ext4    defaults    0 0
UUID=be7101d0-7223-4805-8c0c-d3b656e1f9ea   /mnt/file-16    ext4    defaults    0 0
UUID=33fbf110-2416-4d45-b791-2ebc6fdd5f6c   /mnt/file-17    ext4    defaults    0 0
UUID=2c63d85d-bf20-4859-82b7-d8a942da719f   /mnt/file-18    ext4    defaults    0 0
UUID=9ebfb115-ba06-481e-a961-b183461e38df   /mnt/file-19    ext4    defaults    0 0
UUID=9c65caa8-f3ef-4bfc-89ee-3a7364f51a29   /mnt/file-20    ext4    defaults    0 0
```

Then mount all the shards:

```
for i in {09..20} ; do mkdir /mnt/file-$i ; mount /mnt/file-$i; done
```

Mount an individual shard

Show the connected disk and mount it:

```
$ lsblk | grep 15.6T
sdc       8:32   0 15.6T  0 disk /mnt/file-16

$ mkdir /mnt/file-16
$ mount /dev/sdc /mnt/file-16
```

Fix the permissions

Since the gitaly nodes and the restore nodes have different `passwd` files and therefore different UID's, we need to make them match. `chown -R git /mnt/file-16/git-data` takes an extremely long time for each shard (much longer than it takes to restore the whole disk from backup). If we rebuild the restore node, it would be nice to set the `git` UID to `500` rather than setting the files to the new UID.

Find the UID and GIT of the existing git user and group:

```
grep git /etc/passwd
grep git /etc/group
```

Replace the 998 below with those ID's if different:

```
usermod -u 500 git
groupmod -g 500 git
find /var/opt/gitlab -group 998 -exec chgrp -h git {} \;
find /var/opt/gitlab -xdev -user 998 -exec chown -h git {} \;
find / -xdev -group 998 -exec chown -h git {} \;
find / -xdev -group 998 -exec chgrp -h git {} \;
```

Configure Gitaly

In `gitlab.rb` add this section:

```
git_data_dirs({
        "default" => { "path" => "/var/opt/gitlab/git-data" },
        "nfs-file09" => { "path"=> "/mnt/file-09/git-data" },
        "nfs-file10" => { "path"=> "/mnt/file-10/git-data" },
        "nfs-file11" => { "path"=> "/mnt/file-11/git-data" },
        "nfs-file12" => { "path"=> "/mnt/file-12/git-data" },
        "nfs-file13" => { "path"=> "/mnt/file-13/git-data" },
        "nfs-file14" => { "path"=> "/mnt/file-14/git-data" },
        "nfs-file15" => { "path"=> "/mnt/file-15/git-data" },
        "nfs-file16" => { "path"=> "/mnt/file-16/git-data" },
        "nfs-file17" => { "path"=> "/mnt/file-17/git-data" },
        "nfs-file18" => { "path"=> "/mnt/file-18/git-data" },
        "nfs-file19" => { "path"=> "/mnt/file-19/git-data" },
        "nfs-file20" => { "path"=> "/mnt/file-20/git-data" }
})
```

Then `gitlab-ctl reconfigure`

Start services

Once the reconfigure succeeds, the services need to be started individually.

Make sure postgres is running and in read/write mode by following the steps in an earlier section.

```
gitlab-ctl start gitaly
gitlab-ctl start gitlab-workhorse
gitlab-ctl start redis
gitlab-ctl start puma
gitlab-ctl start nginx
```

Generate admin tokens

If there is not an existing admin token available to authenticate connections with, here is how to generate a new one. Substitute the stuff in all caps for the values you want to use.

```
user = User.find_by_username('YOUR_ADMIN_USER')
token_digest = Gitlab::CryptoHelper.sha256 'PLAIN_TEXT_YOU_MAKE_UP_FOR_THE_TOKEN'
PersonalAccessToken.create!(name: 'Full Access', scopes: [:api], user: user, token_digest: t
```

The `PLAIN_TEXT_YOU_MAKE_UP_FOR_THE_TOKEN` is what you will feed to the `congregate configure` command

Install and configure the congregate tool

The congregate tool is a migration tool maintained by the Professional Services team.

Installation and configuration is described here.

After initializing the tool with `congregate init`, you provide configuration settings in the `data/congregate.conf` file. All the fields are described in the template. Personal Access tokens should be generated on the source and destination instances. The token values are then entered into the config as Base64-encoded to obfuscate them. Obfuscation can be done using `congregate obfuscate`. Example config:

```
[SOURCE]
src_parent_group_path = deletedNamespace
src_parent_group_id = deletedNamespaceID
```

After this, if the tokens and IP's are configured correctly, running `congregate list` should succeed. If it doesn't, check Congregate's logs in `/opt/congregate/data/logs`, or watch the logs on the source server for the reason why (`sudo gitlab-ctl tail gitlab-rails`)

Can't delete the namespace on a test instance?

Have you created a test instance to validate the the restore will work? Is the lack of files on the test instance preventing you from destroying the namespace prior to migrating from the other instance? Rename the namespace instead of deleting it.

```
irb(main):001:0> g = Group.find_by_full_path("deletedgroup")
=> #<Group id:1857667 @ deletedgroup>
irb(main):003:0> g.destroy
Traceback (most recent call last):
       16: from app/models/concerns/storage/legacy_namespace.rb:102:in `block in rm_dir'
       15: from lib/gitlab/gitaly_client/namespace_service.rb:12:in `allow'
       14: from lib/gitlab/temporarily_allow.rb:9:in `temporarily_allow'
       13: from lib/gitlab/gitaly_client/namespace_service.rb:12:in `block in allow'
       12: from app/models/concerns/storage/legacy_namespace.rb:103:in `block (2 levels) in
       11: from lib/gitlab/shell.rb:170:in `mv_namespace'
       10: from lib/gitlab/gitaly_client/namespace_service.rb:40:in `rename'
        9: from lib/gitlab/gitaly_client/namespace_service.rb:46:in `gitaly_client_call'
        8: from lib/gitlab/gitaly_client.rb:169:in `call'
        7: from lib/gitlab/gitaly_client/call.rb:17:in `call'
        6: from lib/gitlab/gitaly_client/call.rb:55:in `recording_request'
        5: from lib/gitlab/gitaly_client/call.rb:18:in `block in call'
        4: from lib/gitlab/gitaly_client.rb:176:in `execute'
        3: from lib/gitlab/gitaly_client.rb:221:in `request_kwargs'
        2: from lib/gitlab/gitaly_client.rb:211:in `authorization_token'
        1: from lib/gitlab/gitaly_client.rb:261:in `token'
RuntimeError (storage not found: "nfs-file11")
irb(main):012:0> g.path
=> "deletedgroup"
```

```
irb(main):013:0> g.path = "renamedgroup"
=> "renamedgroup"
irb(main):017:0> g.save
=> true
irb(main):018:0> g = Group.find_by_full_path("deletedgroup")
=> nil
irb(main):019:0> g = Group.find_by_full_path("renamedgroup")
=> #<Group id:1857667 @ renamedgroup>
irb(main):020:0> exit
```

Some useful commands

To examine the group namespace

From the rails console (`sudo gitlab-rails console`):

```
g = Group.find_by_full_path("interesting-group")
=> #<Group id:1857667 @interesting-group>
irb(main):2:0> g.all_projects.count
=> 158
```

Show the shard and hashed location of the repository files

```
irb(main):4:0> Group.find_by_full_path("interesting-group").all_projects.map.with_index do |
irb(main):5:1*   puts "#{index+1} - #{project.full_path}"
irb(main):6:1>   puts project.repository_storage
irb(main):7:1>   puts project.disk_path
irb(main):8:1> end; nil
```